

CPE 486/586: Deep Learning for Engineering Applications

07 Variational AutoEncoder

Spring 2026

Rahul Bhadani

Outline

1. Motivation
2. AutoEncoder
3. Variational Inference

Motivation

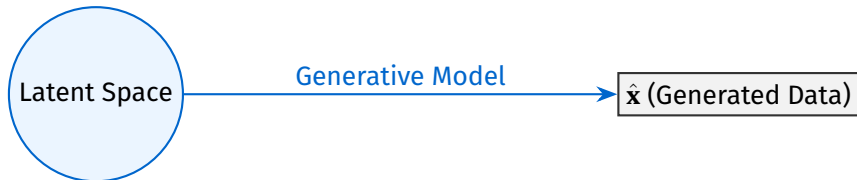
1	0	1	1	0	1	0	0	0	1	1	0	1	1	1	0	1	0	1	1
1	1	1	0	0	1	1	1	1	1	1	0	0	1	0	1	0	0	1	0
1	0	1	0	0	0	1	0	1	0	0	1	0	1	0	1	0	1	1	0

Estimating Probability Distributions for Generative Processes

Given $\mathbf{x}$, we aim to estimate $P(\mathbf{x})$ to provide a complete description of the probabilistic model that generates the observed data.

- ⚡ **Data Synthesis:** We want to learn generative processes capable of producing new datasets.
- ⚡ **Noise Reduction:** By discarding noise, we can more effectively learn underlying physical laws.
- ⚡ **Causality:** Generative models provide a framework to express and understand causal relations.

Example: *If we understand the generative process of wildfires, we can apply that knowledge across different regions, such as California or Australia.*

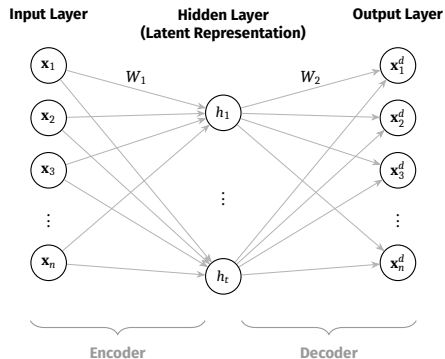


In this lecture, we model generative processes using Variational Autoencoders (VAEs).

AutoEncoder

1	1	1	0	0	0	1	0	1	0	0	0	0	0	1	0	0	1	0	1
0	1	0	0	1	0	0	0	0	0	0	0	0	1	1	1	0	0	0	1
1	0	0	0	1	1	1	1	1	1	1	1	1	0	0	1	0	1	1	0

What is AutoEncoder



Loss function is generally reconstruction loss:

$$\mathcal{L} = \frac{1}{n} \sum_{i=1}^n ||\mathbf{x}_i - \mathbf{x}_i^d||_2^2$$

An **autoencoder** is a neural network, whose main job is to encode the input into a compressed representation, and then decode it back such that the reconstructed input is similar as possible to the original one.

It has three parts:

- 1 **Encoder:** Learns the most relevant features and transfers data into a latent space.
- 2 **Latent representation:** a middle layer that represents feature in some low dimensional space.
- 3 **Decoder:** it takes the reduced features from latent space and lift to the original dimension.

As it turns out, AutoEncoder is a neural network with a bottleneck.

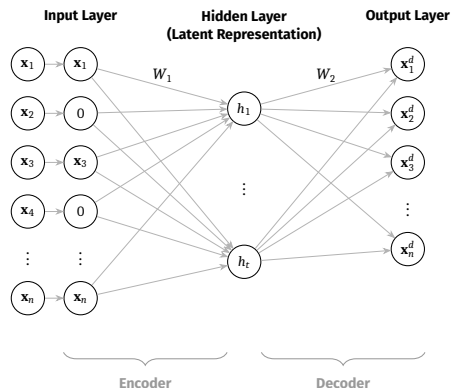
Denoising Autoencoder

An AutoEncoder can end up learning an identity function (also called null function) where input=output.

To remedy, we add some noise to the input to corrupt it before feeding to AutoEncoder. Such an AE is called denoising AutoEncoder (DAE).

There are two ways to add noise to DAE:

- 1 Add Gaussian noise to the input.
- 2 Randomly set some input nodes to zero.
- 3 Masking out a certain percentage of the input's pixels in the case of image data.



Stacked Autoencoders

The stacked autoencoder is a *hack* to obtain a deep model by training only shallow ones in a **layer-wise, greedy fashion**.

- ⚡ **Phase 1:** Train the outermost encoder and decoder as a single shallow AE.
- ⚡ **Phase 2:** Encode the dataset; use the result to train the next inner shallow AE.
- ⚡ **Phase 3:** Repeat until reaching the innermost layers.
- ⚡ **Result:** Stack the pre-trained layers into one deep architecture.

Visualizing the Greedy Training Process

1. Train Outermost (E_1, D_1)



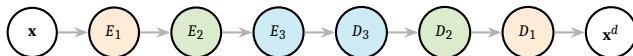
2. Train Middle (E_2, D_2)



3. Train Innermost (E_3, D_3)



4. Final Deep Hacked Stack



Reading List

- 1 Vincent, P., Larochelle, H., Bengio, Y., & Manzagol, P. A. (2008, July). Extracting and composing robust features with denoising autoencoders. In Proceedings of the 25th international conference on Machine learning (pp. 1096-1103). <https://dl.acm.org/doi/pdf/10.1145/1390156.1390294>
- 2 Alain, Guillaume, and Yoshua Bengio. "What regularized auto-encoders learn from the data-generating distribution." The Journal of Machine Learning Research 15, no. 1 (2014): 3563-3593. <https://www.jmlr.org/papers/volume15/alain14a/alain14a.pdf>
- 3 Vincent, Pascal, Hugo Larochelle, Isabelle Lajoie, Yoshua Bengio, Pierre-Antoine Manzagol, and Léon Bottou. "Stacked denoising autoencoders: Learning useful representations in a deep network with a local denoising criterion." Journal of machine learning research 11, no. 12 (2010).
- 4 Zhai, J., Zhang, S., Chen, J. and He, Q., 2018, October. Autoencoder and its various variants. In 2018 IEEE international conference on systems, man, and cybernetics (SMC) (pp. 415-419). IEEE.
- 5 Li, Pengzhi, Yan Pei, and Jianqiang Li. "A comprehensive survey on design and application of autoencoder in deep learning." Applied Soft Computing 138 (2023): 110176.
- 6 Michelucci, Umberto. "An introduction to autoencoders." arXiv preprint arXiv:2201.03898 (2022).
- 7 Bank, Dor, Noam Koenigstein, and Raja Giryes. "Autoencoders." Machine learning for data science handbook: data mining and knowledge discovery handbook (2023): 353-374.

Variational Inference

1	1	1	0	1	0	0	1	1	0	1	0	1	1	1	1	1	0	1
1	1	0	1	0	1	0	1	1	1	1	0	1	1	1	0	1	0	1
0	0	0	1	0	1	0	0	1	0	1	1	0	1	0	1	1	0	1

Motivation

We are interested in **posterior distribution calculation**.

Recall: Bayesian Framework

Given the parameter θ for a data distribution, we have prior as $p(\theta)$, a likelihood, if we write dataset as $\mathcal{D}$, then the posterior distribution can be calculated as:

$$p(\theta|\mathcal{D}) = \frac{p(\mathcal{D}|\theta)p(\theta)}{p(\mathcal{D})}$$

with $p(\mathcal{D}) = \int p(\mathcal{D}|\theta)p(\theta)d\theta$ as the *marginal distribution*.

So, what's the problem?

Motivation

We are interested in **posterior distribution calculation**.

Recall: Bayesian Framework

Given the parameter θ for a data distribution, we have prior as $p(\theta)$, a likelihood, if we write dataset as $\mathcal{D}$, then the posterior distribution can be calculated as:

$$p(\theta|\mathcal{D}) = \frac{p(\mathcal{D}|\theta)p(\theta)}{p(\mathcal{D})}$$

with $p(\mathcal{D}) = \int p(\mathcal{D}|\theta)p(\theta)d\theta$ as the *marginal distribution*.

So, what's the problem?

Computing $p(\mathcal{D})$ is **hard, intractable** due to the volume of data needed, and hence as a result, computing the posterior is hard too.

Variational Inference (VI) is one way to do so.

Variational Objective: The ELBO

We turn VI into an optimization problem: we choose a family of tractable distributions $\mathcal{Q}$ and find a member $q(\theta) \in \mathcal{Q}$ that is as close to $p(\theta|\mathcal{D})$ as possible.

What's the criteria for measuring the closeness between two distributions?

Variational Objective: The ELBO

We turn VI into an optimization problem: we choose a family of tractable distributions $\mathcal{Q}$ and find a member $q(\theta) \in \mathcal{Q}$ that is as close to $p(\theta|\mathcal{D})$ as possible.

What's the criteria for measuring the closeness between two distributions?

KL-Divergence, JS-divergence, etc.

Goal: Minimize KL divergence from $q(\theta)$ to $p(\theta|\mathcal{D})$

$$\text{KL}(q(\theta)||p(\theta|\mathcal{D})) = \mathbb{E}_q[\log q(\theta)] - \mathbb{E}_q[\log p(\theta|\mathcal{D})]$$

Expanding the posterior,

$$\text{KL}(q(\theta)||p(\cdot|\mathcal{D})) = \mathbb{E}_q[\log q(\theta)] - \mathbb{E}_q[\log p(\mathcal{D}|\theta)] - \mathbb{E}_q[\log p(\theta)] + \log p(\mathcal{D})$$

We can rearrange the equations as:

$$\log p(\mathcal{D}) = \text{ELBO}(q) + \text{KL}(q(\theta) \parallel p(\cdot|\mathcal{D}))$$

Variational Objective: The ELBO

The **Evidence Lower Bound (ELBO)** is defined as:

$$\text{ELBO}(q) = \mathbb{E}_q[\log p(\mathcal{D}|\theta)] - \text{KL}(q(\theta) \parallel p(\theta))$$

A $\log p(\mathcal{D})$ is constant wrt q , hence minimizing $\text{KL}(q(\theta) \parallel p(\theta|\mathcal{D}))$ is equivalent to maximizing ELBO. ELBO consists of two terms:

- **Expected log-likelihood:** Encourages q to place mass on parameters that explain the data well.
- **Negative KL to prior:** Encourages q to be close to the prior (regularizer).

VI maximizes a lower bound on the log marginal likelihood.

Mean-field Family

The next question, how do we choose Q . A common choice for the variational family Q is the mean-field family. Other options are *full-covariance Gaussians*, and *Normalizing Flows*.

The mean-field variational family assumes that the latent variables are independent of each other in the approximate posterior. Mathematically, if we have parameters $\theta = (\theta_1, \theta_2, \dots, \theta_M)$, the mean-field approximation factorizes as:

$$q(\theta) = \prod_{j=1}^M q_j(\theta_j)$$

Each q_j is a marginal variational distribution for the j -th parameter or group of parameters. The key point is that there are no cross-terms or dependencies between different θ_j 's in the approximation.

The term "mean-field" comes from physics, specifically from statistical mechanics. In physics, mean-field theory approximates a many-body problem by replacing interactions with an average or "mean" field. Similarly, in variational inference, we're approximating a complex posterior where variables are dependent by a simpler distribution where they're independent.

Stochastic Gradients for VI

We can use gradient descent based methods for optimizing ELBO.

We need to compute gradients of the ELBO wrt to the parameters of q .

$$\mathcal{L}(\lambda) = \text{ELBO}(q_\lambda) = \mathbb{E}_{q_\lambda(\theta)}[\log p(\mathcal{D}|\theta)] - \text{KL}(q_\lambda(\theta) \parallel p(\theta))$$

where λ is the variational parameters such as mean and log-std of a Gaussian.

The KL term can often be computed analytically (e.g., when both q and p are Gaussians). The expected log-likelihood term, however, usually requires approximation.

In PyTorch, we can use reparameterization trick to estimate the gradients of the expectation.